



UNIVERSITÀ DI PARMA

Data Representation

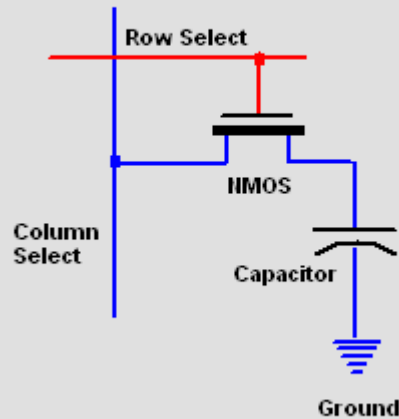
τῷ ἀριθμῷ δὲ τὰ πάντ' ἐπέοικεν
Numero Cuncta Assimilantur

Pitagora (unc.), V sec. BC.

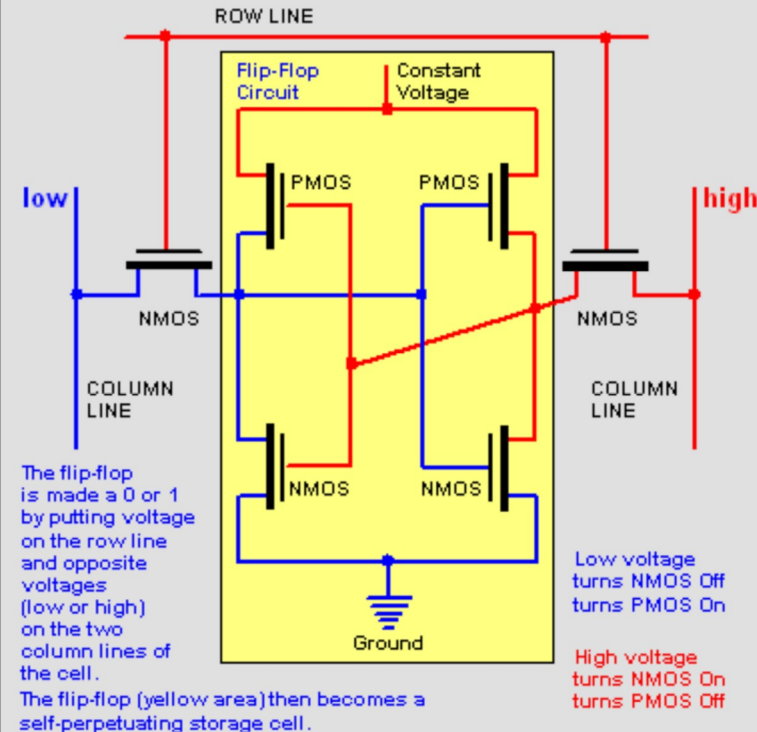
- Memory technologies
- Numbers
 - Representation and Value
 - Decimal Numeral System
 - Binary Numeral System
 - Hexadecimal Numeral System
 - Negative Numbers
 - Non integer Numbers
- Simbols
 - ASCII
 - UNICODE



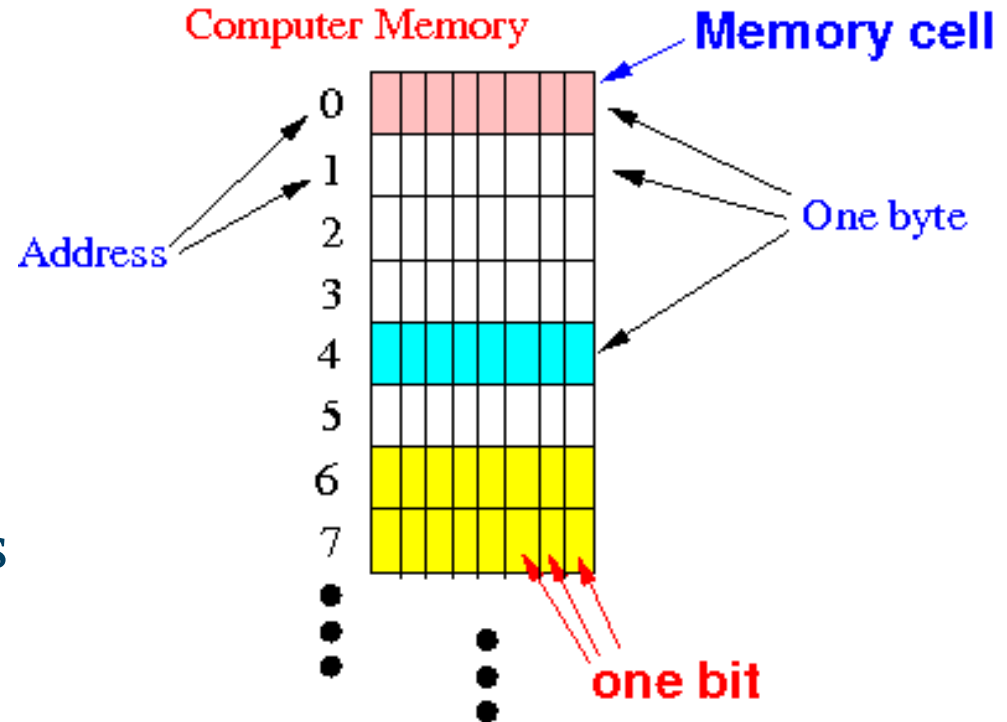
One Dynamic RAM Bit (DRAM bit)



One Static RAM Bit (SRAM bit)



- Data & Code → memory
- How are they stored?
- We have
 - Bytes composed by
 - Bits
- Each Byte has a specific address



- The most basic unit of memory is the **bit**
 - “BInary digiT”
- 2 possible states, formally 0 or 1
- Everything that is stored in computer memory **MUST** be a combination of 2 symbols (0 & 1)
 - True also for other kind of memory supports
- **Then we need to “write” numbers (or symbols) using only sequences of 0 or 1**

THE PHONETIC ALPHABET MORSE CODE GUIDE			
A ALPHA	·--	N NOVEMBER	--·
B BRAVO	---··	O OSCAR	---·
C CHARLIE	--···	P PAPA	·---·
D DELTA	--·	Q QUEBEC	---·-
E ECHO	·	R ROMEO	·-·
F FOXTROT	··-·	S SIERRA	···
G GOLF	--·	T TANGO	-
H HOTEL	····	U UNIFORM	··-
I INDIA	··	V VICTOR	···-
J JULIET	·---	W WHISKEY	·--
K KILO	--·	X X-RAY	··-·
L LIMA	····	Y YANKEE	·-·-
M MIKE	--	Z ZULU	---·



UNIVERSITÀ DI PARMA

Numbers

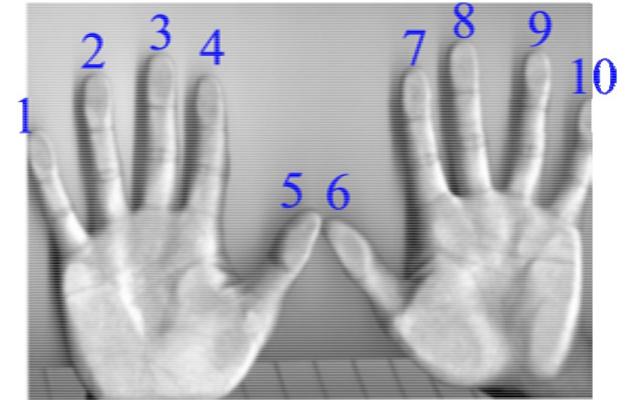
Numbers and their Representations



Numbers and their Representations



- Based on 10 symbols (**base 10**):
 - 0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- Positional system
 - Position is used to weight symbol importance
 - 1786 contains the same symbols as 8761
- Not absolute rules for numeral systems
 - MMXXII
 - Quatre-vingt



- Somewhere in a distant past someone started to mark symbols for numbers
 - But there are so much numbers...
- Let's start to count:
 - 0 1 2 3 4 5 6 7 8 ... and so?
 - We add a second “digit” with a different meaning
 - 30

- We know how to “represent” numbers
- How we **compute** the number given its representation in the decimal numeral system?
- Given the digits d :
 - $d \in \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$
- A number composed by N digits can be **written** as:
 - “ $d_{n-1}d_{n-2}\dots d_2d_1d_0$ ”
 - i.e. 1786
- Its value V can be then **computed** as:
 - $V = d_{n-1} \times 10^{n-1} + d_{n-2} \times 10^{n-2} + \dots + d_2 \times 10^2 + d_1 \times 10 + d_0$
 - Namely as a sum of powers of 10
 - i.e. $1786 = 1 \times 1000 + 7 \times 100 + 8 \times 10 + 6$

- In memory we can have 2 states only $\rightarrow$ formally 1 & 0 “digits”
- We must use base 2 numeral system $\rightarrow$ binary numeral system
- Digits:
 - $b \in \{0, 1\}$
- Representation:
 - $b_{n-1}b_{n-2}\dots b_2b_1b_0$
- Value:
 - $V = b_{n-1} \times 2^{n-1} + b_{n-2} \times 2^{n-2} + \dots + b_2 \times 2^2 + b_1 \times 2 + b_0$

Example

- What is the value of the binary number written as 101110?
- Sum of powers of two:
 - $V = 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2 + 0$
 - $V = 32 + 8 + 4 + 2$

0	0	1	0	1	1	1	0
2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1

- 1) Start from the left namely from the most significant digit
- 2) Init N as 0
- 3) Read current digit
- 4) Multiply N by 2 and sum the digit value
- 5) There are other digits? If so, move to next digit on the right and repeat from step #3
- 6) End of procedure, N contains the value!

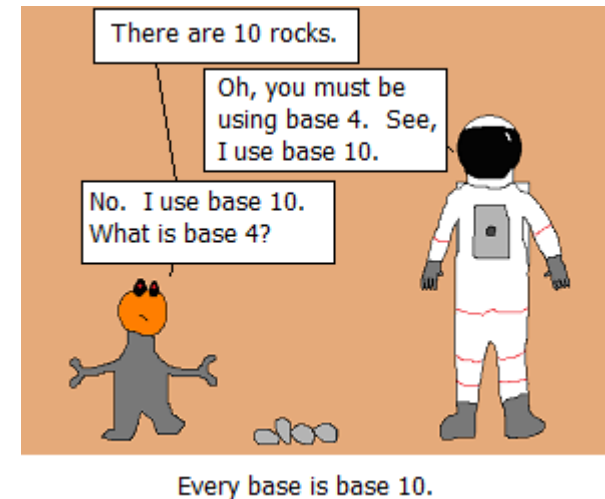
How to compute binary representation

- 1) Consider N as the number we want to know the binary representation
- 2) Init S as an empty sequence of characters
- 3) Divide N by 2
- 4) Put the remainder (0 or 1) in S from right side
- 5) Result is 0?

Yes: S contains the binary representation of value N

No: put $N = \text{remainder}$ and repeat from #3

- Other numeral systems often used are:
 - Hexadecimal representation, base 16:
 - Digits are: 0-9 & A-F
 - 1 digit $\rightarrow$ 4 bits
 - i.e. `0x90FF`
 - Octal representation, base 8:
 - Digits are: 0-7
 - 1 digit $\rightarrow$ 3 bits
- Same conversion rules as before



- From Hexadecimal to Binary:
 - Take bits 4 by 4 (from the least significant ones)
 - Substitute them with the corresponding Hex digit
 - i.e. $1011\ 1001_2 = 0xB9_{16}$
- Practically we can use a “Lookup Table”

Lookup Table



Decimal	Binary	Hexadecimal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

- Practical use when number that are power of 2 are important
- Like binary system but definitely shorter
 - $11011111010 \rightarrow 0x6FA \rightarrow 1786$
 - $16 = 2^4$
- Memory addresses
- Raw byte values
- RGB colors
 - 3 bytes
 - $0xA13A77$

0000000	5025	4644	312d	342e	250a	ecc7	a28f	250a
0000010	4925	766e	636f	7461	6f69	3a6e	6720	2073
0000020	712d	2d20	5364	4641	5245	2d20	4e64	504f
0000030	5541	4553	2d20	4264	5441	4843	2d20	5064
0000040	4644	2041	642d	4f4e	554f	4554	5352	5641
0000050	2045	732d	7250	636f	7365	4373	6c6f	726f
0000060	6f4d	6564	3d6c	6544	6976	6563	4752	2042
0000070	732d	6f43	6f6c	4372	6e6f	6576	7372	6f69

- Same rules as decimal system!
- This is true for each positional numeral system for each base

$$\begin{array}{r} 1\ 0\ 1\ 0\ 1 \\ +\ 1\ 1\ 1\ 0\ 0 \\ \hline \end{array}$$

Bit 0 :

$$\begin{array}{r} 1\ 0\ 1\ 0\ 1 \\ +\ 1\ 1\ 1\ 0\ 0 \\ \hline = \end{array}$$

Bit 1 :

$$\begin{array}{r} 1\ 0\ 1\ 0\ 1 \\ +\ 1\ 1\ 1\ 0\ 0 \\ \hline = \end{array}$$

Bit 2 :

$$\begin{array}{r} 1\ 0\ 1\ 0\ 1 \\ +\ 1\ 1\ 1\ 0\ 0 \\ \hline = \end{array}$$

Carry 1 $1+1=2=10$

Bit 3 :

$$\begin{array}{r} 1\ 0\ 1\ 0\ 1 \\ +\ 1\ 1\ 1\ 0\ 0 \\ \hline = \end{array}$$

Carry 1 $1+1=2=10$

Bit 4 :

$$\begin{array}{r} 1\ 0\ 1\ 0\ 1 \\ +\ 1\ 1\ 1\ 0\ 0 \\ \hline = \end{array}$$

Carry 1 $1+1+1=3=11$

Result

$$\begin{array}{r} 1\ 1 \\ +\ 1\ 0\ 1\ 0\ 1 \\ +\ 1\ 1\ 1\ 0\ 0 \\ \hline = 1\ 1\ 0\ 0\ 0\ 1 \end{array}$$

- We have bits
- Usually they are not addressed as is
- They are then grouped in **words**
 - Typically multiples of 8 (**bytes**)
- Therefore:
 - We have to use $n \times 8$ digits
 - Moreover we have memory limits

word *noun*: the natural unit of data used by a particular architecture design

- A n bit word can represent all integer numbers N such that:
 - $0 \leq N \leq 2^n - 1$
- Given a number N we will need, at least, n bit to represent it in binary format such as:
 - $n > \log_2 N$

- Practical examples:
 - 1 byte namely 8 bits allows to store numbers in the $[0, 255]$ interval
 - 256 different values
- Decimal number 17899_{10} requires, at least, 15 binary digits to be stored ($\log_2(17899)=14,12759..$)
 - Probably we have to use anyway 16 bits namely 2 bytes
 - $17899_{10} = 0100010111101011_2 = 45EB_{16}$

Natural numbers are enough?

- No, they are not enough
- We have also to deal with:
 - Negative numbers
 - Non integer numbers



- Ideally:
 - Same representation for positive numbers
 - Alternative representation for negative numbers
- Moreover, simple:
 - Usage
 - Implementation
- N bit $\rightarrow 2^N$ different permutations
 - 2^N possible numbers

Sign - Magnitude

- One bit is used as sign
 - 0 $\rightarrow$ +
 - 1 $\rightarrow$ -
- Easy to understand (for us!)
- Not so easy to implement
 - That bit has to be separately managed
- Complexity for
 - SW
 - HW

Binario	Decimale
0 000	0
0 001	1
0 010	2
0 011	3
0 100	4
0 101	5
0 110	6
0 111	7
1 000	-0
1 001	-1
1 010	-2
1 011	-3
1 100	-4
1 101	-5
1 110	-6
1 111	-7

Ones' complement

- No special bits
- Positive numbers as usual
- Negative numbers → complement
 - $1 \rightarrow 0$ & $0 \rightarrow 1$
- 8 bit → 255 different values
 - $[-127, 127]$
 - 2 representations for 0

Binario	Decimale
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	-7
1001	-6
1010	-5
1011	-4
1100	-3
1101	-2
1110	-1
1111	-0

Two's Complement

- Used in “modern” architectures
- Positive numbers
 - As usual
- To obtain negative numbers:
 - Take positive
 - Compute ones' complement
 - Add 1
- To obtain negative version from positive number:
 - Sane trick!

Binario	Decimale
0000	0
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	-8
1001	-7
1010	-6
1011	-5
1100	-4
1101	-3
1110	-2
1111	-1

- Range of values for N bits:

- $[-(2^{N-1}), 2^{N-1}-1]$
- For 8 bits $\rightarrow [-128, 127]$

- Advantages

- We do not have +0 e -0
- Better usage of different permutations
- Most significant bit is anyway the sign bit even if it is not the sign bit
- **Same math operations!**

$$\begin{array}{r} 00001001 + \\ 11111011 = \\ \hline 10000100 \end{array}$$

- Easy conversion trick
 - Start from least significant bit
 - Do not change 0s until you find a 1
 - Do not change the first 1 you find
 - Starting from the first 1, complement all other digits!

0110111000 → 1001001000

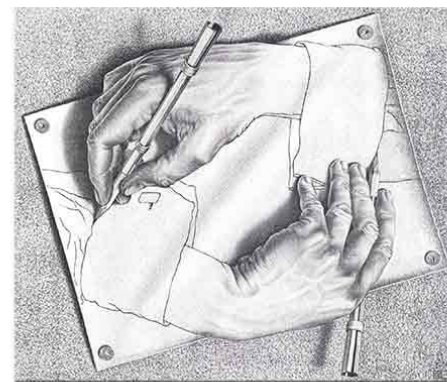
- How do we write in a sheet of paper?
- The same can apply to computer memory
 - At byte level
- When a number span across multiple bytes, what we start to write in memory?
 - Least significant byte?
 - Most significant byte?

Si non nito studere non transiet!

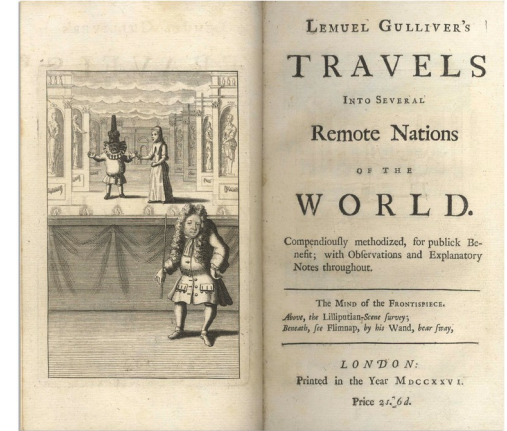
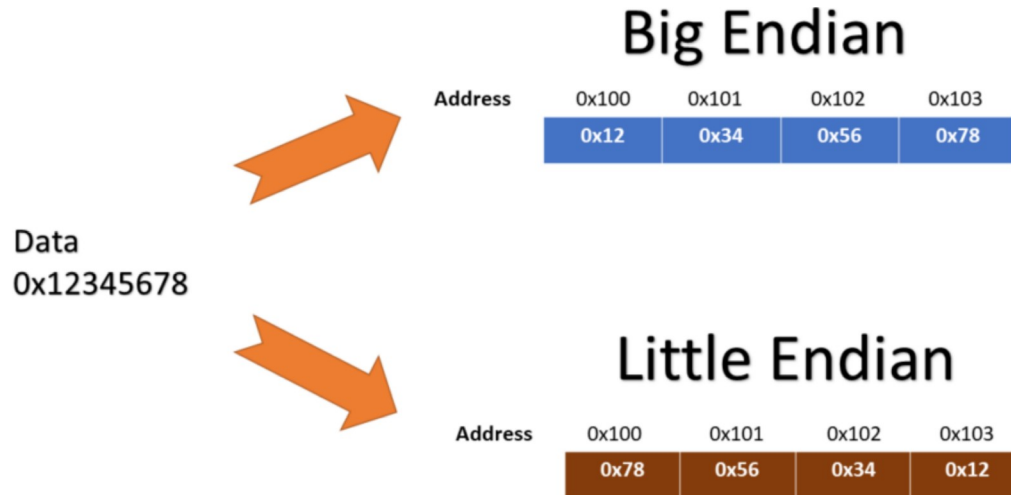
如果我不學習，我就不會通過考試！

אם לא אלמד אני לא אעבור את הבחינה!

إذا لم أدرس فلن أنجح في الامتحان!



- Big endian → More significant bytes corresponds to lower memory addresses
- Little endian → More significant bytes corresponds to higher memory addresses



- Should we care about endianness?
- Yes and **no**
 - In a standalone system we often do not care
 - Issues:
 - Accessing memory on a byte basis
 - Transferring data through a network
 - Using files across different architectures

Natural numbers are enough?

- No, they are not enough
- We have also to deal with:
 - Negative numbers
 - Non integer numbers



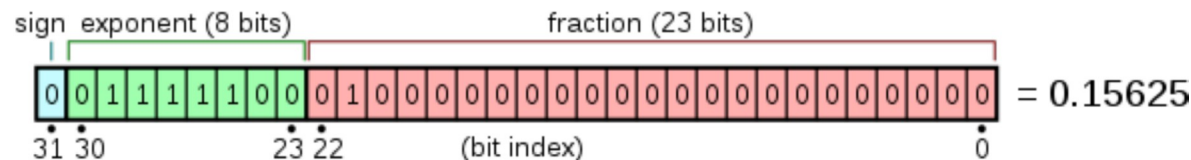
- How they are managed in the decimal numeral system?
 - Example: 123.456
 - The value can be computed as $1 \times 100 + 2 \times 10 + 3 + 4/10 + 5/100 + 6/1000$
 - Decimal numbers can be seen as multiplied by powers of 10
 $V(123,456) = 1 \times 10^2 + 2 \times 10 + 3 + 4 \times 10^{-1} + 5 \times 10^{-2} + 6 \times 10^{-3}$
- Easy extension to the binary numeral system:
 - Example: 1011.1011
 - $V(1011.1011) = 1 \times 2^4 + 0 \times 2^3 + 1 \times 2 + 1 + 1 \times \frac{1}{2} + 0 \times (\frac{1}{2})^2 + 1 \times (\frac{1}{2})^3 + 1 \times (\frac{1}{2})^4$
 - $V(1011.1011) = 1 \times 2^4 + 0 \times 2^3 + 1 \times 2 + 1 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} + 1 \times 2^{-4}$

- A number that has a finite number of digits in decimal format does not necessarily have the same property in binary format.
 - Example: $0.3 \rightarrow 0.01\overline{0011}$
 - And vice versa
- Managing non integer numbers using a computer can lead to **approximations!**
 - i.e. add 0.1 to 0.1 ten times

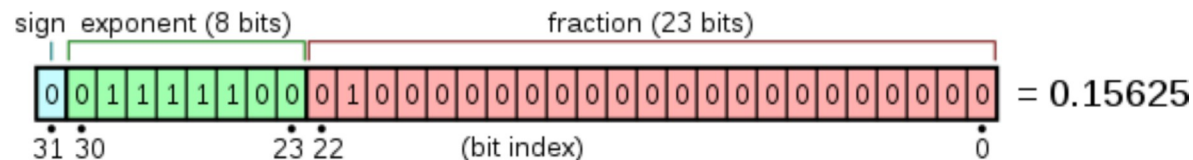
- Also when storing non integer numbers we have to use a limited number of bytes (usually 2, 4, or 8)
- Fixed point binary numbers
 - Given N bits, I use M bits for the fractional part and $N-M$ for the integer portion
- Fixed point data type are seldom used
 - Embedded systems
 - Databases
 - Fonts

- In the decimal numeral system a convenient way to represent numbers is the “Normalized Scientific Notation”
 - $-1234,567 \rightarrow -1.234567 \times 10^3$
- A general format can be seen as:
- (sign) (integer part).(fractional part) $\times 10^{(\text{exponent})}$
 - $0 < (\text{integer part}) < 10$
- Easy to extend to binary system

- IEEE 754 (Standard for Binary Floating-Point Arithmetic)
- From 16 to 128 bits depending on the precision we want to obtain
 - Integer part is always 1 (not necessary to store)
 - Sign bit S
 - Exponent E
 - Fractional part M
 - $N = (-1)^s (1.M) 2^E$
- E & M can have specific values (0,255) to manage special situations (NaN, $\pm\infty$, underflow)

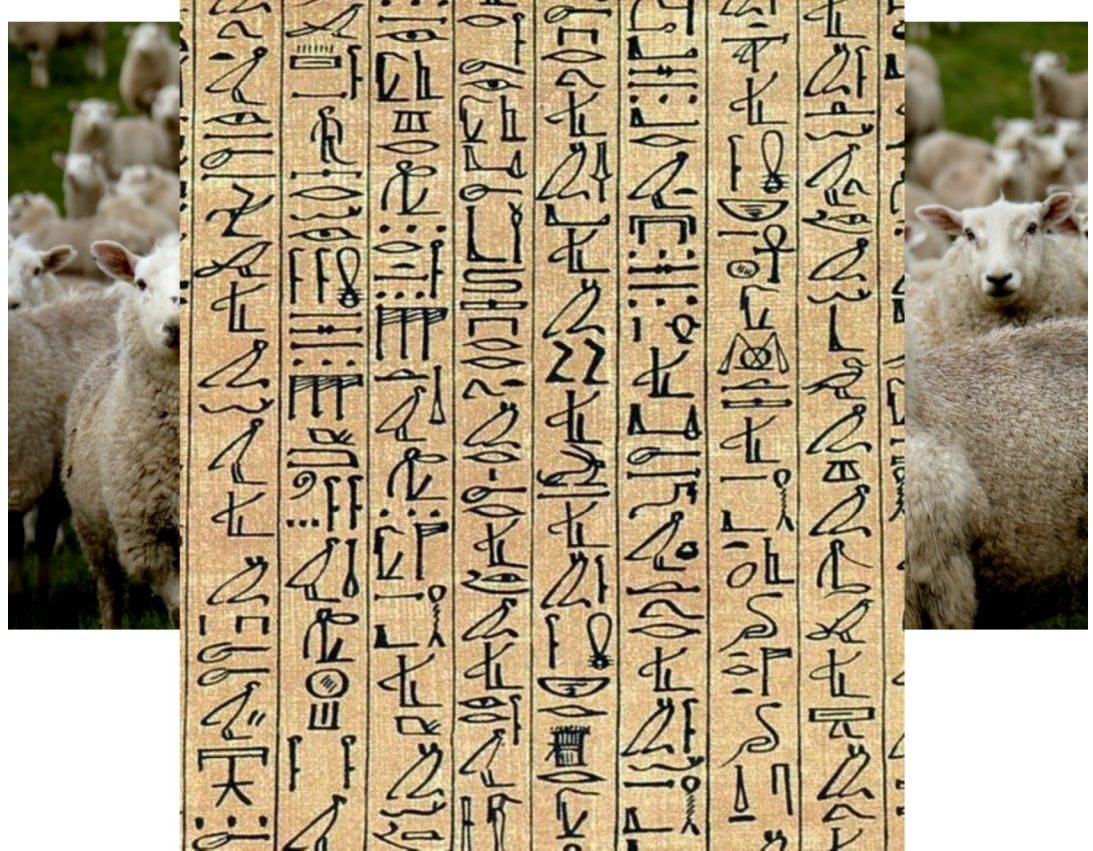


- Ho to decipher that?
 - Sign bit $\rightarrow 0 \rightarrow$ positive number
 - Exponent $\rightarrow 124$
 - We subtract a *bias* (127)
 - $\rightarrow -3$
 - Fractional part $\rightarrow 0.25$ ovvero $(\frac{1}{2})^2$
 - Integer part $\rightarrow 1$
- $-1^0 \times 1.25 \times 2^{-3} \rightarrow 0.15625$



Natural numbers are enough?

- No, they are not enough
- We have also to deal with:
 - Negative numbers
 - Non integer numbers
 - Symbols!



- How to store symbols?
 - We already understood how to store numbers
- We can associate a number to each symbol!
 - Lookup table: Symbols $\rightarrow$ Integer Number
- Some standards
 - ASCII $\rightarrow$ American Standard Code for Information Interchange
 - EBCDIC $\rightarrow$ Extended Binary Coded Decimal Interchange Code
 - UTF-x $\rightarrow$ Unicode Transformation Format

- 7 bit
 - Parity bit for transmission reliability (telegraph)
 - Number range [0, 127]
- 0-31 & 127 → Control symbols
- 32-126 → Printable symbols
- 128-255 → Extended ASCII
 - Mainly for other languages but mostly latin symbols

- In C we are going to have a specific type and syntax to deal with ASCII symbols
 - `char` → i.e. `char a;`
 - A symbol inside single quotation marks → i.e. `'@'`
 - Symbols inside double quotation marks → `"Hello world!"`
- Some issue → i.e. newline
 - Linux → associated to 10
 - Commodores, MAC OS, Apple II → associated to 13
 - Windows → associated to 10 followed by a 13

ASCII control characters		
00	NULL	(Null character)
01	SOH	(Start of Header)
02	STX	(Start of Text)
03	ETX	(End of Text)
04	EOT	(End of Trans.)
05	ENQ	(Enquiry)
06	ACK	(Acknowledgement)
07	BEL	(Bell)
08	BS	(Backspace)
09	HT	(Horizontal Tab)
10	LF	(Line feed)
11	VT	(Vertical Tab)
12	FF	(Form feed)
13	CR	(Carriage return)
14	SO	(Shift Out)
15	SI	(Shift In)
16	DLE	(Data link escape)
17	DC1	(Device control 1)
18	DC2	(Device control 2)
19	DC3	(Device control 3)
20	DC4	(Device control 4)
21	NAK	(Negative acknowl.)
22	SYN	(Synchronous idle)
23	ETB	(End of trans. block)
24	CAN	(Cancel)
25	EM	(End of medium)
26	SUB	(Substitute)
27	ESC	(Escape)
28	FS	(File separator)
29	GS	(Group separator)
30	RS	(Record separator)
31	US	(Unit separator)
127	DEL	(Delete)

ASCII printable characters		
32	space	
33	!	
34	"	
35	#	
36	\$	
37	%	
38	&	
39	'	
40	(	
41	)	
42	*	
43	+	
44	,	
45	-	
46	.	
47	/	
48	0	
49	1	
50	2	
51	3	
52	4	
53	5	
54	6	
55	7	
56	8	
57	9	
58	:	
59	;	
60	<	
61	=	
62	>	
63	?	
64	@	
65	A	
66	B	
67	C	
68	D	
69	E	
70	F	
71	G	
72	H	
73	I	
74	J	
75	K	
76	L	
77	M	
78	N	
79	O	
80	P	
81	Q	
82	R	
83	S	
84	T	
85	U	
86	V	
87	W	
88	X	
89	Y	
90	Z	
91	[	
92	\	
93	]	
94	^	
95	_	
96	`	
97	a	
98	b	
99	c	
100	d	
101	e	
102	f	
103	g	
104	h	
105	i	
106	j	
107	k	
108	l	
109	m	
110	n	
111	o	
112	p	
113	q	
114	r	
115	s	
116	t	
117	u	
118	v	
119	w	
120	x	
121	y	
122	z	
123	{	
124		
125	}	
126	~	

Extended ASCII characters					
128	Ç	160	á	192	Ł
129	à	161	â	193	ł
130	ã	162	ä	194	Ł
131	ä	163	å	195	ł
132	å	164	æ	196	Ł
133	æ	165	ç	197	ł
134	ç	166	¸	198	Ł
135	¸	167	¸	199	ł
136	¸	168	¸	200	Ł
137	¸	169	¸	201	ł
138	¸	170	¸	202	Ł
139	¸	171	¸	203	ł
140	¸	172	¸	204	Ł
141	¸	173	¸	205	ł
142	¸	174	¸	206	Ł
143	¸	175	¸	207	ł
144	É	176	¸	208	ð
145	æ	177	¸	209	Ð
146	Æ	178	¸	210	È
147	ô	179	¸	211	Ê
148	ö	180	¸	212	È
149	ò	181	À	213	Ì
150	û	182	Â	214	Í
151	ù	183	À	215	Î
152	ÿ	184	©	216	Ï
153	Ö	185	¸	217	¸
154	Ü	186	¸	218	¸
155	ø	187	¸	219	¸
156	£	188	¸	220	¸
157	Ø	189	¢	221	¸
158	x	190	¥	222	¸
159	f	191	¸	223	¸
224	Ó				
225	õ				
226	Ô				
227	Õ				
228	ö				
229	Õ				
230	μ				
231	þ				
232	Þ				
233	Ú				
234	Û				
235	Ü				
236	ý				
237	Ý				
238	—				
239	‘				
240	≡				
241	±				
242	≡				
243	¾				
244	¶				
245	§				
246	÷				
247	¸				
248	°				
249	ˆ				
250	˙				
251	¹				
252	ª				
253	º				
254	■				
255	nbsp				

Laboratory PCS are equipped with italian keyboard

How to obtain necessary symbols?

ALT + 3 digits ASCII code

I.e. ALT+123 → “{“

- ASCII is a bit limited: single byte $\rightarrow$ 256 values
- Unicode:
 - Main objective is to enable a single standard to deal with all characters from all scripts
 - Able to deal with 1.114.112 different symbols!
 - Different encodings $\rightarrow$ UTF-x

Name	UTF-8	UTF-16	UTF-32
Code unit size	8 bits	16 bits	32 bits
Fewest bytes per symbol	1	2	4
Most bytes per symbol	4	4	4

- Most popular unicode encoding
- Variable length
 - Space vs speed efficiency
 - 1 byte to map standard English letters and symbols
 - Backward compatible with 7 bits ASCII
 - 2 bytes for additional Latin and Middle eastern symbols
 - 3 bytes for Asian symbols
 - 4 bytes for any other symbol



UNIVERSITÀ DI PARMA

Data Representation

